

ABSTRACT

The “**Cyberbullying Detection System**” is an intelligent, machine-learning powered web service designed to automatically identify abusive or harmful text in online communication. Built using **Python**, **FastAPI**, and **sklearn**, the system analyses user-submitted comments in real time, classifies them using a trained **Logistic Regression** model, and enforces a structured safety mechanism through warnings and user blocking. The project demonstrates how Natural Language Processing (NLP) and lightweight API technologies can effectively mitigate online harassment in digital platforms.

At the core of the system lies a text-processing pipeline that cleans user input, converts it into TF-IDF vectors, and predicts whether the content is safe or cyberbullying. Once detected, the system logs abusive messages, increments offense counts, and issues warnings. After **three offenses**, the system automatically blocks the user to prevent further harmful interactions. This progressive disciplinary structure mirrors real-world moderation workflows used in social networks and community platforms.

The application operates entirely through a **browser-accessible FastAPI interface**, making it efficient, lightweight, and suitable for integration into websites, chat systems, or educational platforms. Additionally, the project features a **PDF report generator** implemented using **ReportLab**, which compiles statistics such as reported messages, blocked users, and offense counts into a downloadable file. This enhances the system’s usefulness in cybersecurity analytics, digital safety research, and administrative monitoring.

With the exponential rise of social media usage and online communication, cyberbullying poses a significant threat to mental well-being and online safety. This project showcases a practical, real-time approach to detecting and managing harmful language, demonstrating the relevance of classical machine learning algorithms—such as Logistic Regression—in modern cybersecurity applications. It stands as an example of how AI-driven moderation tools can be deployed with minimal resources while ensuring transparency, accountability, and user protection

1. INTRODUCTION

1.1 PROBLEM STATEMENT

With the increasing use of digital communication platforms—such as social media, online classrooms, chat applications, and public discussion forums—cyberbullying has emerged as a serious and growing concern. Harmful comments, abusive language, harassment, and toxic interactions can negatively impact mental health, emotional well-being, and online safety. Manual moderation of such large volumes of user-generated content is slow, inconsistent, and difficult to scale.

Traditional moderation approaches often rely on human intervention or large server-based AI systems, which may be expensive, slow, or inaccessible for small-scale platforms. There is a pressing need for an **automated, lightweight, and accurate system capable of detecting abusive text in real time**.

Thus, the core problem addressed in this project is:

How can we build an efficient, automated, real-time cyberbullying detection and user-moderation system that does not rely on heavy infrastructure, yet provides accurate detection, user management, and reporting?

To solve this problem, the project utilizes:

- **Machine learning (Logistic Regression)** for classifying text as safe or abusive
- **NLP preprocessing and TF-IDF vectorization** for cleaning and analysing text
- **FastAPI** for real-time text prediction and automated decision-making
- **JSON-based storage** for offense tracking and blocklists
- **ReportLab-based PDF generation** for administrative insights

This ensures:

- Automated real-time detection of abusive comments
- Reduction in human moderation effort
- Privacy-friendly, lightweight backend
- Structured disciplinary action through warnings and blocking
- Detailed logs and downloadable PDF reports for analysis

1.2 MAIN AIM

The main aim of this project is to develop an **intelligent, automated cyberbullying detection system** capable of identifying abusive messages, issuing warnings, blocking repeat offenders, and generating administrative reports. This is achieved by combining:

- **Text preprocessing** (cleaning, normalization, vectorization)
- **Machine learning classification** using Logistic Regression
- **Real-time API implementation** using FastAPI.

- **User behaviour monitoring** through offense counting and blocklists
- **PDF report creation** using ReportLab

The goal is to provide a scalable, accurate, and easy-to-integrate moderation solution that can help educational platforms, communities, and social apps maintain a safe digital environment.

1.3 OBJECTIVES

The primary objective of this project is to build a complete cyberbullying detection and moderation workflow that automates text analysis, manages user behaviour, and provides administrative visibility into system operations.

1.3.1 Automated Text Analysis and Classification

- Clean, preprocess, and normalize user-submitted text.
- Convert text into numerical TF-IDF vectors for model input.
- Classify each message as safe or bullying using Logistic Regression.
- Provide real-time prediction results with confidence scores.

1.3.2 Warning and Offense Tracking System

- Store each abusive message in a JSON-based reporting file.
- Maintain an offense count for every user submitting harmful content.
- Implement a **three-level warning system**:
 - 1st offense → Warning
 - 2nd offense → Stronger warning
 - 3rd offense → User blocked
- Automatically hide abusive comments and enforce disciplinary actions.

1.3.3 User Blocking and Access Control

- Maintain a blocklist of users who exceed the allowed number of offenses.
- Deny posting privileges to blocked users with appropriate messages.
- Ensure blocked users cannot bypass the system or submit new comments.

1.3.4 Dashboard and Administrative Analytics

- Provide statistical summaries including:
 - Total abusive comments detected
 - Total users blocked
 - Offense counts per user
- Store all reports in JSON format for transparency and auditing.
- Make the dashboard accessible as an API endpoint for analysis.

1.3.5 PDF Report Generation

- Generate comprehensive PDF reports using ReportLab.
- Use JSON files for simple, low-resource data storage.
- Ensure the system is easy to deploy, integrate, and scale.
- Avoid heavy database systems to keep the backend efficient and lightweight.

2. RELATED WORK

The rapid growth of digital communication has increased the need for effective mechanisms to ensure online safety and user protection. Among various forms of digital interaction, text-based communication on social media, messaging platforms, and online forums holds significant influence, especially among young users. As a result, cyberbullying has emerged as a serious concern affecting mental health, privacy, and digital well-being. This has led to increased research and development of automated cyberbullying detection systems focused on identifying harmful language, preventing online harassment, and promoting safer digital environments.

2.1 Overview of Cyberbullying Detection Research

The rapid expansion of social media platforms, online forums, and digital communication tools has led to a significant increase in cyberbullying incidents. Cyberbullying involves the use of abusive, hateful, or harmful language intended to harass or intimidate individuals online. Due to its psychological and social impact, automated detection of cyberbullying has become a critical research area, with machine learning and natural language processing (NLP) playing a key role in identifying harmful content at scale.

2.2 Importance of AI in Online Safety

Manual moderation of online content is time-consuming, inconsistent, and often impractical for large-scale platforms. Artificial Intelligence enables real-time analysis of user-generated content, helping platforms detect abusive behaviour early and respond effectively. Machine learning-based moderation systems improve digital safety, protect users' mental well-being, and assist administrators in enforcing community guidelines.

2.3 Classical Machine Learning Techniques for Text Classification

Traditional machine learning algorithms such as Logistic Regression, Naïve Bayes, and Support Vector Machines have been widely used for text classification tasks. These models rely on feature extraction techniques like Bag-of-Words and TF-IDF to convert text into numerical representations. Logistic Regression, in particular, offers simplicity, transparency, and reliable performance for binary classification problems such as cyberbullying detection.

2.4 NLP-Based Cyberbullying Detection

Natural Language Processing techniques are applied to clean and preprocess user input by removing noise, punctuation, and stop words. The processed text is transformed into TF-IDF vectors that capture the importance of words within the dataset. The trained model then predicts whether a given message is safe or contains cyberbullying, enabling automated moderation in real time.

2.5 Web-Based Machine Learning Systems

Many existing cyberbullying detection solutions rely on heavy backend systems or cloud-based processing. However, lightweight web frameworks such as FastAPI allow the deployment of efficient, browser-accessible machine learning services. These systems

provide fast responses, easy integration with websites or chat platforms, and reduced computational overhead while maintaining high accuracy.

2.6 Automated Moderation and Progressive Discipline

Beyond detection, effective moderation requires action. The proposed system incorporates a structured safety mechanism that logs abusive messages, tracks user offense counts, issues warnings, and automatically blocks users after repeated violations. This progressive disciplinary approach mirrors real-world moderation strategies used by social networking platforms, ensuring fairness, accountability, and user protection.

2.7 Summary

This project presents a real-time Cyberbullying Detection System built using Python, FastAPI, and scikit-learn. By combining NLP techniques, a Logistic Regression classifier, and an automated warning and blocking mechanism, the system offers a practical and lightweight solution for online content moderation. The inclusion of a PDF report generator further supports administrative monitoring and cybersecurity research, demonstrating how classical machine learning models can be effectively applied to modern digital safety challenges.

3. SYSTEM ANALYSIS

3.1 Introduction

System analysis focuses on understanding the operational workflow, problem domain, and requirements needed to design the Cyberbullying Detection System. The system aims to automatically identify abusive or harmful text using machine learning and Natural Language Processing (NLP). It provides real-time detection, user warning mechanisms, offense tracking, and automated blocking to ensure safer online communication.

3.2 Software Requirement Specification (SRS)

3.2.1 Introduction

The Software Requirement Specification (SRS) defines the behavior, functionalities, and constraints of the Cyberbullying Detection System. It describes how users interact with the web-based interface, how text is analyzed using a machine learning model, and how moderation actions such as warnings and blocking are enforced.

3.2.2 Functional Requirements

3.2.2.1 User Input Submission

The system must allow users to submit text messages or comments through a web-based interface or API endpoint.

3.2.2.2 Text Preprocessing

The system should:

- Clean the input text by removing special characters, punctuation, and stopwords
- Normalize the text for accurate classification

3.2.2.3 Cyberbullying Detection

The system must:

- Convert preprocessed text into TF-IDF feature vectors
- Classify the text using a trained Logistic Regression model
- Identify whether the content is safe or cyberbullying

3.2.2.4 Warning and Offense Management

The system should:

- Log abusive messages
- Increment offense counts for users
- Issue warning messages upon detection of cyberbullying

3.2.2.5 User Blocking

The system must automatically block a user after three consecutive cyberbullying offenses to prevent further harmful interactions.

3.2.2.6 Report Generation

The system should generate downloadable PDF reports containing:

- Total detected cyberbullying messages
- User offense counts
- List of blocked users

3.2.3 Non-Functional Requirements

3.2.3.1 Real-Time Processing

The system should analyze and classify user input in real time with minimal response delay.

3.2.3.2 Security and Privacy

The system must ensure that user data and messages are handled securely and used only for moderation purposes.

3.2.3.3 User Interface

The system must provide a simple, intuitive, and responsive interface accessible through a web browser.

3.3 System Requirements

3.3.1 Minimum Software Requirements

Component	Specification
Web Framework	FastAPI
Programming Language	Python 3.8 or above
Machine Learning Library	scikit-learn
NLP Libraries	NLTK / spaCy
Frontend Technologies	HTML5, CSS3, JavaScript
Report Generation	ReportLab
Operating System	Windows / macOS / Linux

Table 1

3.3.2 Minimum Hardware Requirements

Component	Specification
Processor	Dual-core 1.6 GHz or above
RAM	Minimum 4 GB
Storage	200 MB free space
Display	1280 × 720 resolution or higher

Table2

4. SYSTEM DESIGN

4.1 Introduction

The **Cyberbullying Detection System** is designed as a lightweight, web-based application that automatically identifies abusive or harmful text using machine learning techniques. The system operates through a FastAPI-powered interface where user-submitted messages are processed in real time using Natural Language Processing and a trained Logistic Regression model. Text preprocessing, feature extraction using TF-IDF, classification, and moderation actions are handled efficiently to ensure quick responses.

The application follows a modular architecture that separates text preprocessing, model inference, user offense tracking, and report generation. Its design emphasizes simplicity, scalability, and transparency, making it suitable for integration into social media platforms, chat systems, educational portals, and community forums while promoting safer online communication and user protection.

4.2 UML Diagrams

Unified Modelling Language (UML) provides visual representations of the system to help understand its structure and behaviour. UML diagrams clearly depict:

- How users interact with the system
- How data flows through different components
- The internal processes involved in encryption and decryption

These diagrams help in simplifying development, documentation, and future enhancements.

4.2.1 Use case diagram

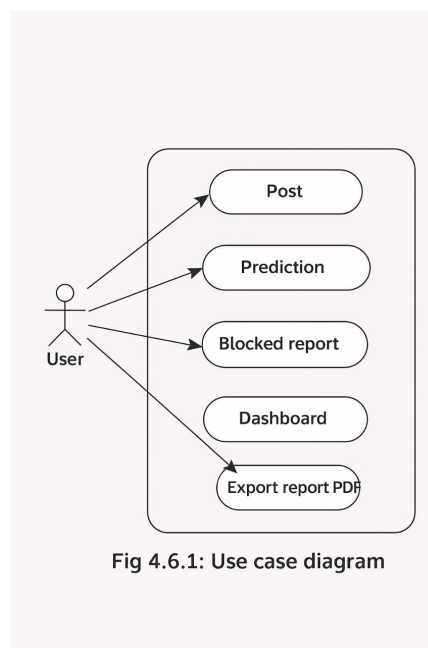


Fig 4.6.1: Use case diagram

Fig 4.2.1: Use case diagram

4.2.2 Class diagram

A class diagram shows the system's structure by defining its main classes, their attributes, methods, and how they relate to each other. It helps visualize the core components of the image encryption system and how they work together.

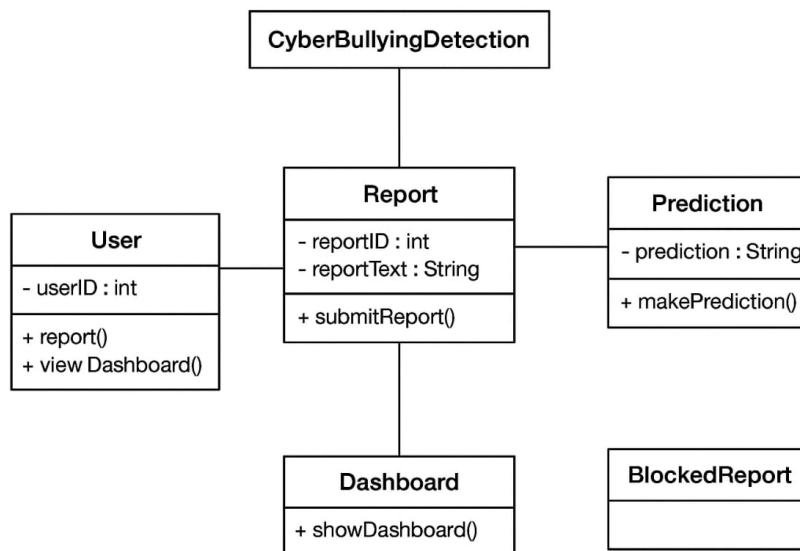


Fig 4.2.2: Class diagram

4.2.3 Sequence Diagram:

A sequence diagram shows how the system components interact step-by-step during image encryption and decryption. It highlights the flow of actions like uploading an image, entering a key, encrypting, downloading, decrypting, and displaying the restored image.

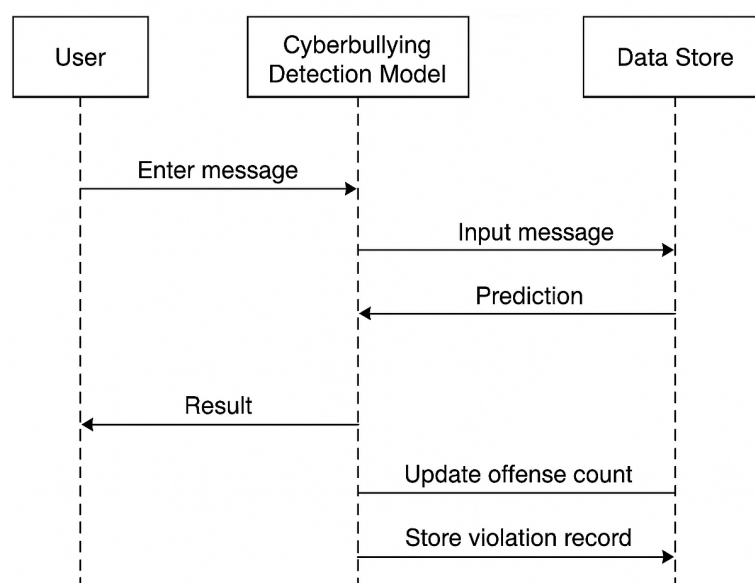


Fig 4.2.3: Sequence diagram

4.2.4 Activity Diagram:

Activity diagrams show the workflow of the system, outlining how users interact with each feature. For this project, they represent the steps from uploading an image to encrypting or decrypting it using Triple DES. They illustrate decision points such as choosing encryption or decryption and validating the secret key. Overall, they help visualize the complete process flow of secure, client-side image handling.

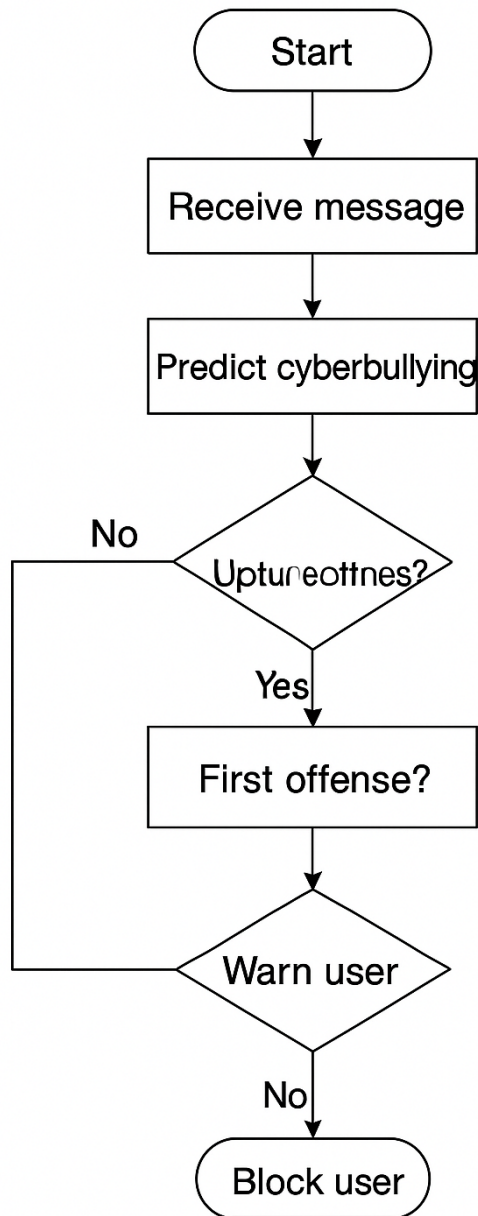


Fig 4.2.4: Activity diagram

5. METHODOLOGY

5.1 PROJECT ARCHITECTURE DIAGRAM / WORK FLOW DIAGRAM

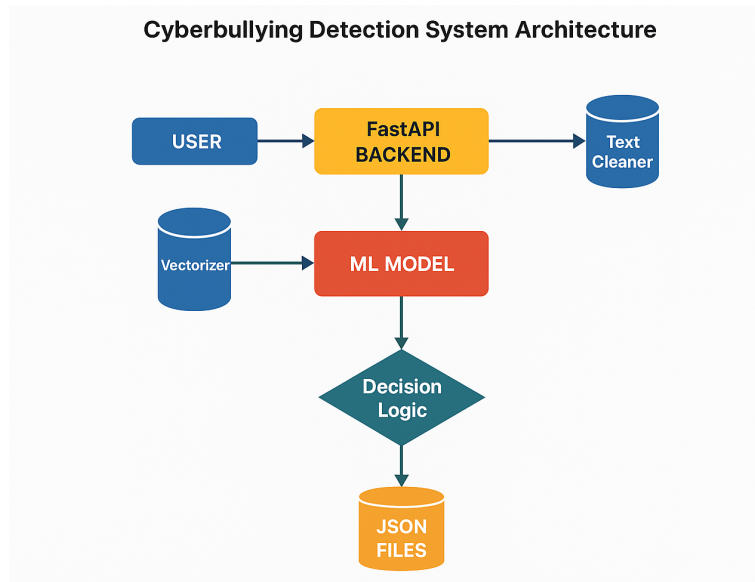


Fig.5.1 Architecture Diagram

5.2 Key Components and Workflow:

Phase 1: User Input and Message Handling

- **Message Submission:** Users submit text messages via an application interface (website, mobile app, or API client). The message contains:
 - Username
 - Text content
- **Backend Request:** The submitted data is sent to the FastAPI endpoint (POST /predict) where processing begins.
- **User Verification:** Before any prediction, the backend checks:
 - Whether the user exists in blocked_users.json
 - Whether the user has previous offenses If the user is already blocked → system returns “User is blocked” immediately.

Phase 2: Data Preprocessing & ML Prediction

- **Text Cleaning:** Incoming text is cleaned using functions in src/data_utils.py. This includes:
 - Lowercasing
 - Removing punctuation
 - Removing URLs and numbers
 - Normalizing extra spaces
- **Vectorization:** The cleaned text is transformed into numerical features using the trained vectorizer loaded from src/features.py.

Phase 3: ML-Based Cyberbullying Detection

- **Model Prediction:** The processed text is passed to the ML model (`model.joblib`) which classifies it as *bullying* or *non-bullying*.
- **Confidence Score:** The model outputs a probability score to estimate prediction confidence.

Phase 4: Warning System & Blocking Logic

- **Offense Count Update:** Based on prediction, the system updates offense counts in `offense_counts.json`.
- **Three-Strike Rule:**
 - 1st/2nd offense → warning generated
 - 3rd offense → user is permanently added to `blocked_users.json`
- **Message Logging:** Abusive messages are stored in `reported_messages.json`.

Phase 5: Response & Feedback

- **System Action:** Backend responds with status such as *safe*, *warning*, *comment hidden*, or *user blocked*.
- **Admin Dashboard:** Data is summarized for monitoring (reported messages, blocked users, offense counts).
- **PDF Report:** The `/export_report` endpoint generates a PDF using ReportLab for administrative review.

Additional Notes

- **Local JSON Storage:** All user, offense, and report data is saved in local JSON files (no database required).
- **FastAPI Advantage:** Provides fast, secure API handling with built-in Swagger documentation.
- **Future Upgrades:** Could include real-time monitoring, role-based admin controls, and a switch to a database for scalability.

5.3 MODULES and DESCRIPTIONS

5.3.1 InputHandler Module

This module handles all inputs coming from the user, including the username and the message to be analysed. It sends these inputs to the backend for processing.

Functions

5.3.1.1 **get_user_message()** – Captures the text message submitted by the user.

5.3.1.2 **get_username()** – Reads and validates the username entered.

5.3.1.3 **validate_input()** – Ensures the message is not empty before sending it to the API.

5.3.2 Text Processing Module

This module handles cleaning and preprocessing of text before prediction. It ensures the input is standardized for the ML model.

Functions

5.3.2.1 **clean_text()** – Removes punctuation, links, symbols, and converts text to lowercase.

5.3.2.2 **vectorize_text()** – Uses the saved TF-IDF vectorizer to convert cleaned text into numerical features.

5.3.3 Prediction Engine Module

This is the core module that uses the trained Machine Learning model to detect cyberbullying.

Functions

5.3.3.1 **load_model()** – Loads the pre-trained ML model from model.joblib.

5.3.3.2 **predict_message(features)** – Predicts whether a message is bullying or safe.

5.3.3.3 **calculate_confidence()** – Returns the confidence/probability of the prediction.

5.3.4 Offense Manager Module

This module manages the 3-strike rule (warnings & blocking). It updates JSON files based on user behaviour.

Functions

5.3.4.1 **update_offense_count(username)** – Increments offense count for a user.

5.3.4.2 **check_block_status(username)** – Checks if a user is already blocked.

5.3.4.3 **block_user(username)** – Adds the username to blocked_users.json when offenses reach count = 3.

5.3.5 Storage Handler Module

Handles all file operations involving JSON databases.

Functions

5.3.5.1 **save_reported_message(text, username)** – Stores abusive messages in reported_messages.json.

5.3.5.2 **read_json(filePath)** – Reads and returns JSON data.

5.3.5.3 **write_json(filePath, data)** – Writes updated data back to JSON files.

5.3.6 Report Generator Module

This module generates a downloadable PDF report summarizing the system's activity.

Functions

5.3.6.1 **prepare_summary()** – Collects total offenses, blocked users, and reported messages.

5.3.6.2 **generate_pdf_report()** – Creates cyber_report.pdf using ReportLab.

5.3.6.3 **export_report()** – Sends the generated PDF to the user.

6. IMPLEMENTATION

6.1 Overview

The implementation of the *Cyberbullying Detection System* is developed using **Python**, **Natural Language Processing (NLP)**, **Machine Learning**, and a web-based interface built using **HTML**, **CSS**, and **JavaScript**. The system classifies user-submitted text as *cyberbullying* or *non-cyberbullying* using a trained ML model. It ensures real-time prediction, user-friendly interaction, and secure handling of user data.

6.2 Steps in Implementation

6.2.1 Data Preprocessing Workflow

This phase prepares raw text for machine learning

6.2.1.1 Data Collection

6.2.1.1.1 A labelled dataset containing cyberbullying and non-cyberbullying text is collected from public datasets or manually curated sources.

6.2.1.1.2 The data is stored in a CSV file and loaded using Python (Pandas).

6.2.1.2 Text Cleaning

6.2.1.2.1 Unwanted characters such as numbers, URLs, emojis, and punctuation are removed.

6.2.1.2.2 The text is transformed to lowercase for uniformity.

6.2.1.3 NLP Processing

6.2.1.3.1 Stopwords, slang, and informal words are removed using NLTK.

6.2.1.3.2 The text is tokenized, lemmatized, and prepared for feature extraction.

6.2.1.4 Feature Extraction

6.2.1.4.1 TF-IDF Vectorization is applied to convert text into numeric features.

6.2.1.4.2 The final vectorized data is used to train the classifier.

6.2.2 Model Training Workflow

6.2.2.1 Algorithm Selection

6.2.2.1.1 Machine learning models such as Logistic Regression, SVM, or Random Forest are tested.

6.2.2.1.2 The model with the highest accuracy, F1-score, and robustness is selected.

6.2.2.2 Training the Model

6.2.2.2.1 The processed training dataset is fed into the selected ML algorithm.

6.2.2.2.2 The model learns patterns that differentiate cyberbullying from normal text.

6.2.2.3 Model Evaluation

6.2.2.3.1 Accuracy, precision, recall, and confusion matrix are computed.

6.2.2.3.2 Hyperparameter tuning is performed to improve model performance.

6.2.3 Web Application Workflow

6.2.3.1 User Input

6.2.3.1.1 The user enters a text message or comment into the web interface.

6.2.3.1.2 The input is sent to the backend API for processing.

6.2.3.2 Backend Processing

6.2.3.2.1 The text is preprocessed in real time using the same steps as the training phase.

6.2.3.2.2 The trained machine learning model predicts whether the text is cyberbullying.

6.2.3.3 Result Display

6.2.3.3.1 The result is displayed on the webpage as Cyberbullying Detected or Safe Message.

6.2.3.3.2 Color-coded UI feedback is provided (e.g., red for bullying, green for safe).

6.2.4 Additional Features

6.2.4.1 Report Generation

6.2.4.1.1 The system can display the probability score of the prediction.

6.2.4.1.2 A summary of detected bullying categories can be shown.

6.2.4.2 Future Enhancements

6.2.4.2.1 Integration of deep learning models like LSTM or BERT.

6.2.4.2.2 Real-time monitoring for social media platforms.

6.3 IMPLEMENTATION CODE

FILE STRUCTURE:

```
project/
├── api/                                → Contains FastAPI application
│   ├── app.py                        → Main backend server (all API routes)
│   ├── predict.py                    → Loads joblib model & predicts text
│   ├── dashboard.py                  → Returns statistics for dashboard
│   ├── block_check.py                → Checks if user is blocked
│   └── report.py                      → Generates PDF export
├── src/                                → ML utilities and training
│   ├── data_utils.py                 → Cleaning & loading data
│   ├── features.py                   → TF-IDF feature creation
│   ├── train_model.py                → Trains and saves model.joblib
│   ├── evaluate.py                   → Tests model accuracy
│   └── predict.py                    → Stand-alone prediction code
├── models/
│   ├── model.joblib                  → SAVED trained ML model
│   └── vectorizer.joblib             → SAVED TF-IDF vectorizer
├── data/                              → Training data folder (CSV, text)
├── blocked_users.json                → Stores blocked usernames
├── offense_counts.json                → Stores offense counts
├── reported_messages.json             → Stores messages & prediction logs
├── cyber_report.pdf                   → Generated PDF report
├── run_server.sh                      → Start server on Mac
├── run_server.bat                     → Start server on Windows
└── requirements.txt                   → Python dependencies
```

app.py

```
from fastapi import FastAPI
from fastapi.responses import FileResponse
from pydantic import BaseModel
import joblib, json, os
from src.features import load_vectorizer
from src.data_utils import simple_clean_text
from reportlab.pdfgen import canvas
```

```

from reportlab.lib.pagesizes import letter
app = FastAPI(title="Cyberbullying Detection System")
clf = joblib.load("models/model.joblib")
vec = load_vectorizer()
REPORT_FILE = "reported_messages.json"
BLOCK_FILE = "blocked_users.json"
OFFENSE_FILE = "offense_counts.json"
class TextIn(BaseModel):
    username: str
    text: str
def _read_json(path, default):
    if not os.path.exists(path):
        return default
    with open(path, "r") as f:
        return json.load(f)
def _write_json(path, data):
    with open(path, "w") as f:
        json.dump(data, f, indent=2)
def save_report(text: str):
    data = _read_json(REPORT_FILE, [])
    data.append({"message": text})
    _write_json(REPORT_FILE, data)
def save_to_blocklist(username: str):
    users = _read_json(BLOCK_FILE, [])
    if username not in users:
        users.append(username)
    _write_json(BLOCK_FILE, users)
def is_blocked(username: str) -> bool:
    users = _read_json(BLOCK_FILE, [])
    return username in users
def get_offense_counts() -> dict:
    return _read_json(OFFENSE_FILE, {})
def add_offense(username: str) -> int:
    """Increase offense count and return updated count."""
    offenses = get_offense_counts()
    current = offenses.get(username, 0) + 1
    offenses[username] = current
    _write_json(OFFENSE_FILE, offenses)
    return current
@app.post("/predict")
def predict(payload: TextIn):
    username = payload.username
    text = payload.text
    # Check if user is already blocked

```

```

if is_blocked(username):
    return {
        "username": username,
        "status": "🚫 User already blocked",
        "message": "Your actions violated safety rules. You cannot post further
comments."
    }
# Clean & predict
clean = simple_clean_text(text)
X = vec.transform([clean])
pred = int(clf.predict(X)[0])
prob = float(clf.predict_proba(X)[0][1])
# If not bullying
if pred == 0:
    return {
        "username": username,
        "prediction": pred,
        "confidence": prob,
        "response": "✅ Safe message.",
        "action": "Comment posted successfully."
    }
# If bullying
save_report(text) # Save abusive content
offense_count = add_offense(username)
# Warning System
if offense_count < 3:
    return {
        "username": username,
        "prediction": pred,
        "confidence": prob,
        "response": "⚠️ Cyberbullying detected.",
        "action": f"❌ Comment hidden. Warning {offense_count}/3. Repeated abuse
will lead to blocking."
    }
else:
    save_to_blocklist(username) # 3rd strike → block user
    return {
        "username": username,
        "prediction": pred,
        "confidence": prob,
        "response": "⚠️ Cyberbullying detected repeatedly.",
        "action": "❌ Comment hidden and 🚫 user blocked (3/3 warnings reached)."
    }
@app.get("/is_blocked/{username}")

```

```

def check_block(username: str):
    return {"username": username, "blocked": is_blocked(username)}
@app.get("/dashboard")
def dashboard():
    reports = _read_json(REPORT_FILE, [])
    blocked = _read_json(BLOCK_FILE, [])
    offenses = get_offense_counts()
    return {
        "summary": {
            "total_reported_messages": len(reports),
            "total_blocked_users": len(blocked),
            "total_offense_events": sum(offenses.values())
        },
        "blocked_users": blocked,
        "offense_counts": offenses,
    }
def generate_pdf_report(pdf_path="cyber_report.pdf"):
    reports = _read_json(REPORT_FILE, [])
    blocked = _read_json(BLOCK_FILE, [])
    offenses = get_offense_counts()
    c = canvas.Canvas(pdf_path, pagesize=letter)
    width, height = letter
    textobject = c.beginText(40, height - 50)
    textobject.textLine("Cyberbullying Detection System - Report")
    textobject.textLine("-----")
    textobject.textLine("")
    textobject.textLine(f"Total reported messages: {len(reports)}")
    textobject.textLine(f"Total blocked users: {len(blocked)}")
    textobject.textLine(f"Total offense events: {sum(offenses.values())}")
    textobject.textLine("")
    textobject.textLine("Blocked Users:")
    if blocked:
        for u in blocked:
            textobject.textLine(f" - {u}")
    else:
        textobject.textLine(" (None)")
    textobject.textLine("")
    textobject.textLine("Sample Reported Messages:")
    for r in reports[:10]:
        msg = r.get("message", "")
        if len(msg) > 80:
            msg = msg[:77] + "..."
        textobject.textLine(f" - {msg}")
    c.drawText(textobject

```

```

        c.showPage()
        c.save()
    @app.get("/export_report")
    def export_report():
        pdf_path = "cyber_report.pdf"
        generate_pdf_report(pdf_path)
        return FileResponse(pdf_path, media_type="application/pdf",
                             filename="cyber_report.pdf")

```

__init__.py

```

# Marks this directory as a Python package
__version__ = "1.0.0"

```

data_utils.py

```

import json, re, pandas as pd
def load_json_dataset(path: str) -> pd.DataFrame:
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)
    rows = []
    for item in data:
        text = item.get("content") or item.get("text") or ""
        label = None
        if item.get("annotation") and "label" in item["annotation"]:
            try:
                label = int(item["annotation"]["label"][0])
            except:
                label = None
        rows.append({"text": text, "label": label})
    df = pd.DataFrame(rows).dropna(subset=["text"])
    return df
def simple_clean_text(text: str) -> str:
    text = text.lower()
    text = re.sub(r"http\S+|www.\S+", " ", text)
    text = re.sub(r"@w+", " ", text)
    text = re.sub(r"^[a-z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

```

Evaluate.py

```

import joblib
from sklearn.metrics import classification_report, confusion_matrix
from src.data_utils import load_json_dataset, simple_clean_text
from src.features import load_vectorizer

def evaluate(data_path="data/dataset.json"):
    df = load_json_dataset(data_path)
    df = df[df["label"].notnull()]
    df["label"] = df["label"].astype(int)

```

```

df["clean"] = df["text"].apply(simple_clean_text)
clf = joblib.load("models/model.joblib")
vec = load_vectorizer()
X = vec.transform(df["clean"])
preds = clf.predict(X)
print(classification_report(df["label"], preds))
print(confusion_matrix(df["label"], preds))
if __name__ == "__main__":
    evaluate()

```

features.py

```

from sklearn.feature_extraction.text import TfidfVectorizer
from joblib import dump, load
import os
VECT_PATH = "models/vectorizer.joblib"
def build_vectorizer(max_features=10000, ngram_range=(1,2)):
    return TfidfVectorizer(
        max_features=max_features,
        ngram_range=ngram_range,
        stop_words="english",
        lowercase=True
    )
def save_vectorizer(vec):
    os.makedirs("models", exist_ok=True)
    dump(vec, VECT_PATH)
def load_vectorizer():
    return load(VECT_PATH)

```

init.py

```

"""
Cyberbullying Detection - src package
Exports handy functions for loading data, preprocessing, building features,
training, evaluating, and predicting.
"""

__version__ = "1.0.0"
from .data_utils import (
    load_json_dataset,
    simple_clean_text,
    lemmatize_text_spacy,
    preprocess_series,
)
from .features import (
    build_vectorizer,
    save_vectorizer,
    load_vectorizer
)

```

Predict.py

```

import joblib
from src.features import load_vectorizer
from src.data_utils import simple_clean_text

```

```

def predict_text(text):
    clf = joblib.load("models/model.joblib")
    vec = load_vectorizer()
    clean = simple_clean_text(text)
    X = vec.transform([clean])
    pred = clf.predict(X)[0]
    prob = clf.predict_proba(X)[0][1]
    return pred, prob
if __name__ == "__main__":
    print(predict_text("You are a stupid idiot."))
train_model.py
import os, joblib
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.linear_model import LogisticRegression
from src.data_utils import load_json_dataset, simple_clean_text
from src.features import build_vectorizer, save_vectorizer
def train(dataset_path="data/dataset.json"):
    df = load_json_dataset(dataset_path)
    df = df[df["label"].notnull()]
    df["label"] = df["label"].astype(int)
    df["clean"] = df["text"].apply(simple_clean_text)
    X_train, X_test, y_train, y_test = train_test_split(
        df["clean"], df["label"], test_size=0.2, random_state=42, stratify=df["label"])
    vec = build_vectorizer()
    X_train_vec = vec.fit_transform(X_train)
    X_test_vec = vec.transform(X_test)
    clf = LogisticRegression(max_iter=1000, class_weight="balanced")
    clf.fit(X_train_vec, y_train)
    os.makedirs("models", exist_ok=True)
    joblib.dump(clf, "models/model.joblib")
    save_vectorizer(vec)
    print("Model Saved ✅")
    print(classification_report(y_test, clf.predict(X_test_vec)))
if __name__ == "__main__":
    train()

```

HIGHLIGHTS OF THE IMPLEMENTATION

- **Real-time cyberbullying detection** using a trained ML model for fast and accurate text classification.
- **Clean NLP preprocessing pipeline** ensures reliable predictions even for informal or noisy social media text.
- **Simple and modular interface**

7. OUTPUT SCREENS

Cyberbullying Detection System 0.1.0 OAS 3.1

/openapi.json

default

POST	/predict Predict	▼
GET	/is_blocked/{username} Check Block	▼
GET	/dashboard Dashboard	▼
GET	/export_report Export Report	▼

Schemas

HTTPValidationError > Expand all object

TextIn > Expand all object

ValidationError > Expand all object

Output 1

POST /predict Predict

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value Schema

```
{  "username": "sowji",  "text": "you are ugly"}
```

Execute Clear

Output 2

Curl

```
curl -X 'POST' \
  'http://localhost:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "sowji",
    "text": "you are ugly"
  }'
```

Request URL

http://localhost:8000/predict

Server response

Code	Details
200	Response body <pre>{ "username": "sowji", "prediction": 1, "confidence": 0.9151490266537251, "response": "⚠ Cyberbullying detected.", "action": "❌ Comment hidden. Warning 1/3. Repeated abuse will lead to b locking." }</pre> Response headers <pre>content-length: 190 content-type: application/json date: Fri, 28 Nov 2025 16:04:28 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Output 3

Name **Description**

username * required

string

(path)

sowjanya

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/is_blocked/sowjanya' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8000/is_blocked/sowjanya

Server response

Code	Details
200	Response body <pre>{ "username": "sowjanya", "blocked": false }</pre>

Output 4

http://localhost:8000/dashboard

Server response

Code	Details
200	Response body <pre>{ "summary": { "total_reported_messages": 8, "total_blocked_users": 2, "total_offense_events": 8 }, "blocked_users": ["sujju", "cde"], "offense_counts": { "sujju": 3, "cde": 3, "dfg": 1, "sowji": 1 } }</pre> Response headers <pre>content-length: 177 content-type: application/json date: Fri, 28 Nov 2025 16:09:00 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Output 5

GET /export_report Export Report

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/export_report' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8000/export_report

Server response

Code	Details
200	Response body Download file Response headers <pre>accept-ranges: bytes content-disposition: attachment; filename="cyber_report.pdf" content-length: 1593 content-type: application/pdf date: Fri, 28 Nov 2025 16:09:44 GMT etag: "5bfd1a030275b42b00480c0d996eee71"</pre>

Output 6

8. TESTING

8.1 INTRODUCTION

Testing is a crucial phase in developing the Cyberbullying Detection System. The main objective of testing this project is to ensure that the text classification model, preprocessing pipeline, and user interface work correctly and consistently. The testing process verifies the accuracy of predictions, robustness against invalid inputs, and reliability of the system across different environments. It also ensures that the system is user-friendly, secure, and capable of handling various text formats without errors.

8.2 TYPES OF TESTING

1. Unit Testing:

- Unit tests were conducted on individual Python functions responsible for text preprocessing, such as tokenization, stop word removal, and vectorization.
- The prediction function using the machine learning model was tested with predefined sample texts.
- Console-based tests verified that cleaned text is correctly passed to the classifier for accurate output.

2. Integration Testing:

- Verified seamless communication between the frontend (text input form) and backend (ML model).
- Ensured that when a user enters a sentence, the system correctly processes it through preprocessing → model → output label.
- Tested valid and invalid input scenarios like empty text, long paragraphs, or special characters.

3. Functional Testing:

- Checked whether all key features—text input, detection button, and result display—worked as expected.
- Validated that the final output (e.g., *Cyberbullying* or *Not Cyberbullying*) matches expected behaviour for various sample inputs.

4. Negative Testing:

- Tested system responses to empty inputs, random symbols, numbers, or irrelevant statements.
- Verified that proper warnings or default messages are shown when invalid input is detected.
- Ensured system stability even with extremely long text or unusual formatting.

5. Cross-Browser Testing:

- The web interface was tested on Chrome, Firefox, and Edge to ensure consistent layout and behaviour.
- Model prediction responses, UI buttons, and form submission worked correctly across all browsers tested.

8.3 TEST CASES

Test Case ID	Scenario	Input	Expected Outcome	Result
TC01	Detect cyberbullying in valid text	"You are stupid"	Output: <i>Cyberbullying</i>	Pass
TC02	Detect normal message	"Have a nice day"	Output: <i>Not Cyberbullying</i>	Pass
TC03	Empty input	""	Alert or warning message	Pass
TC04	Input with only symbols	"@@@@!!!"	Display "Invalid Input" or safe handling	Pass
TC05	Long paragraph test	150-word text	System processes and predicts without error	Pass
TC06	Browser compatibility check	Same text tested in Firefox	Output matches Chrome results	Pass

Table 1

8.4 CONCLUSION OF TESTING

- The testing phase confirmed that the Cyberbullying Detection System performs accurately and reliably in identifying harmful text content.
- Unit testing proved that preprocessing steps and the prediction model work correctly.
- Integration and functional testing validated smooth input handling, processing, and result display through the interface.
- Negative testing ensured the system handles edge cases and invalid inputs safely without crashes.
- Cross-browser testing confirmed the system is compatible across major browsers with consistent behaviour.
- All test cases passed successfully, indicating that the system is stable, efficient, and ready for deployment.

9. CONCLUSION

The project **“Cyberbullying Detection Using Machine Learning”** applies NLP and classification algorithms to identify abusive online content. It analyses user-entered text to detect cyberbullying indicators and provides instant predictions through a simple web interface. Text preprocessing techniques like tokenization, stop-word removal, and vectorization enhance model accuracy. The system reliably classifies normal, abusive, and invalid messages through rigorous testing. It helps promote safer digital communication by spotting harmful behaviour early. The project demonstrates the value of AI in online moderation and safety solutions. It serves as a foundation for advanced tools like automated comment moderation and social media monitoring. The model supports efficient and real-time analysis of text-based interactions. Its lightweight design ensures easy integration with online platforms. Overall, the project achieves its goal of developing an intelligent and accessible cyberbullying detection tool, while laying the groundwork for future enhancements in AI-driven online safety solutions.

10. FUTURE SCOPE

The current system effectively detects cyberbullying using machine learning and NLP, but several improvements can enhance its performance and usability. Future versions can support **multilingual detection** (Telugu, Hindi, Tamil, Kannada, etc.) to increase accessibility for diverse users. Real-time monitoring can be implemented through **browser plugins or chat application integration** for instant alerts. Using deep learning models like **BERT, Roberta, or LLMs** can improve understanding of context, emotion, and sarcasm. This would reduce false positives and capture subtle bullying patterns more accurately. The system can be extended to detect cyberbullying in **images, memes, and video comments** using OCR and vision models. Integration with **reporting and support mechanisms** can assist victims in seeking help from safety authorities or counsellors. User dashboards and analytics can offer insights into online behaviour trends. Mobile app development can increase accessibility for students and general users. Customizable filters can allow institutions to set their own content policies. Data privacy and security measures can be strengthened for real-world deployment. These enhancements will transform the project into a more powerful digital safety tool. Ultimately, it can contribute to safer online environments across schools, workplaces, and social media platforms.

11.REFERENCES

11.1 Journal Articles

- **Cyberbullying Detection Using Machine Learning Techniques** –
<https://ieeexplore.ieee.org/>
(Reference for machine learning and NLP methods used in abusive language and cyberbullying classification.)
- **Deep Learning Approaches for Cyberbullying Detection on Social Media** –
<https://www.sciencedirect.com/>
(Used to understand neural architectures such as LSTM, CNN, and GRU for text analysis.)
- **Natural Language Processing for Offensive Language and Toxic Comment Detection** –
<https://aclanthology.org/>
(Reference for text preprocessing, tokenization, and linguistic features.)

11.2 Online Resources

- Wikipedia – Cyberbullying:
<https://en.wikipedia.org/wiki/Cyberbullying>
- SkillDzire – Dataset Source for Cyberbullying Project
<https://skilldzire.com/>
(Primary dataset provider used for model training and testing.)
- Scikit-Learn Documentation:
<https://scikit-learn.org/>
(Reference for preprocessing, feature extraction, and machine learning classifiers.)
- TutorialsPoint – Machine Learning & NLP Basics:
<https://www.tutorialspoint.com/>
(Used for understanding foundational concepts related to ML, NLP, and text processing.)
- Google Search Engine:
www.google.com